

算法题基础知识测试答案

一休老师的算法题复习建议：今年线上线下的算法课都由我来给大家讲，预计：①线下课打基础（本次考试有不熟悉的同学需要上）→②~⑤线上强化课专项练习→⑥线下课带练及提高。我的目标是所有同学都能拿下70%的分数，进度快的同学可以做出大部分题目的满分方法，请大家放心，只需要两三周的练习就可以实现这一点。

请完成顺序表、链表、树、图的数据结构定义，* 表示暂时可以忽略

```
#define MAXV 100 // 元素或顶点个数
typedef int ElemType; // ElemType表示int类型
```

在遍历时请使用 `visit(...)` 函数指代对于某结点/元素的处理，括号内填入对应结点/元素。考试中不同题目主要是通过修改该函数来做答。

顺序表

顺序表已给出，考试中推荐都使用数组写法。

数组写法

数据结构定义

```
ElemType data[MAXV];
```

遍历

```
void traverse(ElemType data[], int n)
{
    for (int i=0; i<n; i++)
        visit(data[i]);
}
```

顺序查找

```
int Seq_Search(ElemType data[], int n, ElemType x)
{
```

```
for (int i = 0; i < n; i++)
    if (data[i] == x)
        return i;
return -1;
}
```

* 折半查找

```
// 折半查找
int B_Search(int A[], int L, int R, int x)
{
    int mid;
    while (L <= R)                // 只要区间不为空就继续
    {
        mid = (R + L) / 2;        // 求中点
        if (A[mid] == x)         // 查找成功
            return mid;
        else
            if (x < A[mid])
                R = mid - 1;      // x 在左半区间, L ~ mid-1
            else L = mid + 1;     // x 在右半区间, mid+1 ~ R
    }
    return -1;                   // 查找失败
}
```

结构体写法

数据结构定义

```
typedef struct
{
    ElemType data[MAXV];
    int length;                // 顺序表长度
} SqList;
```

遍历

```
void traverse(SqList S)
{
    for (int i=0; i<S.length; i++)
        visit(S.data[i]);
}
```

链表

单链表

数据结构定义

```
typedef struct LNode
{
    ElemType data;
    struct LNode *next;
} LNode;
LNode *head; // 头指针
```

遍历

```
void traverse(LNode *L)
{
    LNode *p = L->next;
    while (p != NULL)
    {
        visit(p->data);
        p = p->next;
    }
}
```

树与二叉树

二叉链表表示法

数据结构定义

```
typedef struct BiTNode
{
    ElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *linklist;
```

遍历——先序（根 → 左 → 右）

```
void PreOrder(BiTNode *T)
{
    if (T == NULL) return;
    visit(T);
    PreOrder(T->lchild);
    PreOrder(T->rchild);
}
```

遍历——中序（左 → 根 → 右）

```
void InOrder(BiTNode *T) {
    if (T == NULL) return;
    InOrder(T->lchild);
    visit(T);
    InOrder(T->rchild);
}
```

遍历——后序（左 → 右 → 根）

```
void PostOrder(BiTNode *T)
{
    if (T == NULL) return;
    PostOrder(T->lchild);
    PostOrder(T->rchild);
    visit(T);
}
```

孩子兄弟表示法

数据结构定义

```
typedef struct TNode
{
    ElemType data;
    struct TNode *firstchild, *nextbro;
} TNode;
TNode *root;
```

* 遍历——先序遍历修改（不需要记）

```
void PreOrder(TNode *T)
{
    if (T == NULL) return;
    visit(T);
    PreOrder(T->firstchild);
    PreOrder(T->nextbro);
}
```

* 遍历——中序遍历修改（不需要记）

```
void InOrder(TNode *T)
{
    if (T == NULL) return;
    InOrder(T->firstchild);
    visit(T);
    InOrder(T->nextbro);
}
```

* 遍历——后序遍历修改（不需要记）

```
void PostOrder(TNode *T)
{
    if (T == NULL) return;
    PostOrder(T->firstchild);
    PostOrder(T->nextbro);
    visit(T);
}
```

双亲表示法

数据结构定义

常用写法：

```
typedef struct
{
    PTNode nodes[MAXV];
    int n; // 结点数
} PTree;
```

不用写法：

```
typedef struct
{
    ElemType data;
    int parent; // 双亲下标, -1 表示根
} PTNode;
PTNode nodes[MAXV];
```

遍历所有结点

```
void traverse(PTree T)
{
    for (int i = 0; i < T.n; i++)
        visit(T.nodes[i].data);
}
```

二叉排序树

数据结构定义同二叉链表表示法，只是保证权值：左<中<右

查找

```
BiTNode* BST_Search(BiTNode *T, ElemType x)
{
    BiTNode *p = T;
    while (p != NULL)
    {
        if (x == p->data)
            return p;
    }
}
```

```

    if (x < p->data)
        p = p->lchild;
    else
        p = p->rchild;
}
return NULL;
}

```

图

邻接矩阵

数据结构定义

```

typedef struct                                // 图的定义
{
    int numVertices, numEdges;                // 图中实际的顶点数和边数
    char VerticesList[MAXV];                 // 可选：每个顶点名称
    int Edge[MAXV][MAXV];                   // 邻接矩阵
}MGraph;

```

遍历所有顶点和边——循环

```

void traverse(MGraph G)
{
    for (int i = 0; i < G.numVertices; i++)
        for (int j = 0; j < G.numVertices; j++)
            visit(G.Edge[i][j]);             // 无权图 0 表示无边
}

```

邻接表

数据结构定义

```

// 边结点
typedef struct ArcNode
{
    int adjvex;                                // 邻接点在顶点表中的
    下标
    struct ArcNode *next;                     // 指向下一条边
}

```

```

    int weight; // 有权图可选：边权值
} ArcNode;

// 顶点结点
typedef struct VNode
{
    char data; // 可选：顶点信息
    ArcNode *first; // 指向第一条邻接边
} VNode;

// 邻接表图定义
typedef struct
{
    VNode VerticesList[MAXV]; // 顶点表
    int numVertices, numEdges; // 顶点数、边数
} ALGraph;

```

遍历所有顶点和边——循环

```

void traverse(ALGraph G)
{
    for (int i = 0; i < G.numVertices; i++)
    {
        ArcNode *p = G.VerticesList[i].first;
        while (p != NULL)
        {
            visit(p); // 这里处理的是边
            p = p->next;
        }
    }
}

```

* 遍历——DFS

```

bool visited[MAXV] = {false};
void DFS_ALGraph(ALGraph G, int v)
{
    visit(v); // 访问顶点
    visited[v] = true;
    ArcNode *p = G.VerticesList[v].first;
    while (p != NULL)
    {

```



```

        if (!visited[p->adjvex])
            DFS_ALGraph(G, p->adjvex);
        p = p->next;
    }
}

```

* 遍历——BFS

```

bool visited[MAXV] = {false}; // 表示顶点是否已入队
void BFS_ALGraph(ALGraph G, int v)
{
    int queue[MAXV], front = 0, rear = 0;

    // 1. 将 v 入队, 标记
    queue[rear++] = v;
    visited[v] = true;

    // 2. 如果队列不为空则循环
    while (front < rear)
    {
        // 2-a. 选择队头元素 u 访问
        int u = queue[front++];
        visit(u);

        // 2-b. 将 u 出发可到达的所有未访问顶点加入队列
        ArcNode *p = G.VerticesList[u].first;
        while (p != NULL)
        {
            if (visited[p->adjvex] != true)
            {
                queue[rear++] = p->adjvex;
                visited[p->adjvex] = true;
            }
            p = p->next;
        }
    }
}

```